

# Capa

---

**Curso:** [Nome do Curso]

**Disciplina:** [Nome da Disciplina]

**Nome do Software:** New Gym App

**Integrantes:**

- [Nome Completo do Integrante 1]
  - [Nome Completo do Integrante 2]
- 

## 1. Escopo

---

O New Gym App é um sistema de gerenciamento de academia, voltado para personal trainers e alunos, com foco em cadastro, acompanhamento e edição de treinos e exercícios. O objetivo é digitalizar o controle de treinos, alunos e exercícios, proporcionando atualização em tempo real e facilidade de uso.

**Telas desenvolvidas:**

- Tela de Login
- Tela de Cadastro de Usuário (Personal/Aluno)
- Tela Inicial (Home)
- Tela de Listagem de Alunos (para Personal)
- Tela de Cadastro de Aluno
- Tela de Detalhe do Aluno (visualização dos treinos)
- Tela de Cadastro de Exercício
- Tela de Biblioteca de Exercícios
- Tela de Edição de Exercício
- Tela de Cadastro de Treino
- Tela de Edição de Treino
- Tela de Perfil do Usuário
- Tela de Logout

**Obs.:** Não serão desenvolvidas telas de pagamento, relatórios financeiros, integração com dispositivos externos ou funcionalidades de chat.

---

## 2. Requisitos

---

### 2.1 Requisitos Funcionais

- RF01: O sistema deve permitir o cadastro de usuários com papéis de Personal Trainer e Aluno.
- RF02: O Personal Trainer pode cadastrar, editar e remover alunos.
- RF03: O Personal Trainer pode cadastrar, editar e remover exercícios.
- RF04: O Personal Trainer pode criar, editar e remover treinos para cada aluno.

- RF05: O Aluno pode visualizar seus treinos e exercícios atribuídos.
- RF06: O sistema deve atualizar as informações em tempo real (streaming).
- RF07: O sistema deve permitir login e logout de usuários.
- RF08: O sistema deve validar dados obrigatórios nos formulários.

## 2.2 Requisitos Não Funcionais

- RNF01: O sistema deve ser desenvolvido em Flutter (cross-platform).
- RNF02: O backend deve utilizar Firebase (Firestore e Auth).
- RNF03: O sistema deve ser responsivo e funcionar em dispositivos móveis e web.
- RNF04: O sistema deve garantir a segurança dos dados dos usuários.
- RNF05: O sistema deve apresentar tempo de resposta inferior a 2 segundos para operações CRUD.
- RNF06: O código deve seguir boas práticas de organização e arquitetura.

---

## 3. Stack Tecnológica

- **Frontend:** Flutter 3.38.2 (Dart 3.10.0)
- **Gerenciamento de Estado:** Riverpod
- **Backend:** Firebase (Firestore, Auth)
- **Navegação:** go\_router
- **Controle de Versão:** Git/GitHub
- **IDE:** VS Code / Android Studio

---

## 4. Paleta de Cores

| Cor        | Código  | Preview                                                                                     |
|------------|---------|---------------------------------------------------------------------------------------------|
| Primária   | #1976D2 |  #1976D2 |
| Secundária | #64B5F6 |  #64B5F6 |
| Fundo      | #F5F5F5 |  #F5F5F5 |
| Ação       | #43A047 |  #43A047 |
| Erro       | #D32F2F |  #D32F2F |

---

## 5. Arquitetura do Software

O projeto adota arquitetura modular baseada em features, com separação em camadas:

- **Presentation:** Telas e providers (Riverpod)
- **Domain:** Modelos de dados
- **Service:** Serviços de acesso ao Firebase
- **Shared:** Widgets e utilitários reutilizáveis

Organização por pastas:

```
lib/  
├─ core/ (config, models, services, shared_widgets, utils)  
└─ features/ (auth, students, manage_exercises, etc.)
```

O gerenciamento de estado é feito com Riverpod, e a atualização dos dados é reativa via Streams do Firestore.

---

## 6. Projeto do Banco de Dados

---

### Firestore (NoSQL):

- **users** (coleção)
  - id: string
  - name: string
  - email: string
  - role: string (personal/aluno)
  - personalTrainerId: string (se aluno)
- **exercises** (coleção)
  - id: string
  - name: string
  - category: string
  - instructions: string
- **workouts** (coleção)
  - id: string
  - name: string
  - studentId: string
  - exercises: array de objetos {exerciseId, series, reps, notes}

Relacionamentos:

- Um personal pode ter vários alunos.
- Um aluno pode ter vários treinos.
- Um treino pode ter vários exercícios.

---

## 7. Testes de Software

---

- **Testes de Unidade:**
  - Testes para validação de modelos e serviços.
- **Testes de Integração:**
  - Testes de fluxo de cadastro, login, criação de treino e exercício.
- **Testes Manuais:**

- Validação de navegação, responsividade e atualização em tempo real.
- **Cobertura:**
  - [Descrever se há uso de ferramentas de cobertura, ex: `flutter test --coverage`]

---

## 8. Build

---

- **Comando de build para web:** `flutter build web`
- **Comando de build para Android:** `flutter build apk`
- **Comando de build para iOS:** `flutter build ios`
- **Pré-requisitos:**
  - Flutter instalado
  - Firebase configurado (`firebase_options.dart` gerado localmente)
  - Dependências instaladas (`flutter pub get`)

---

## 9. Planejamento de Release

---

| Data       | Entrega | Descrição                                                                  |
|------------|---------|----------------------------------------------------------------------------|
| 10/11/2025 | N2      | Autenticação, cadastro/listagem de alunos, cadastro/listagem de exercícios |
| 18/11/2025 | N3      | Cadastro/edição de treinos, perfil, ajustes finais, testes e build         |
| 25/11/2025 | Final   | Entrega final do app completo                                              |

---

### Observação:

Adapte os nomes dos integrantes, curso e disciplina conforme sua turma. Para gerar o PDF, use um editor de texto, ajuste a diagramação conforme ABNT, e exporte para PDF.